

Hyperparameter Optimization and Loss-Curve Diagnosis of a Keras Text Classifier for Customer Sentiment and Intent

Ram Kelly Jersen Leliza
College of Computer Studies and Engineering
Jose Rizal University
Mandaluyong City, Philippines
ramkellyjersen.leliza@my.jru.edu

Abstract—This paper reports seven controlled TensorFlow/Keras training runs for a four-class customer feedback classifier, benchmarked against the no-code Machine Learning for Kids model built in Exercise PT-P1 (90.00% accuracy, 0.9095 weighted F1). An Embedding + GlobalAveragePooling1D + Dense architecture was trained under deliberately induced underfitting (validation loss 1.3826, statistically indistinguishable from the $\ln(4) = 1.3863$ random-guess floor) and overfitting (validation loss 7.6743 at unchanged accuracy). Four separate configurations converged on an identical 70.83% validation accuracy ceiling, establishing that neither capacity nor regularization was the binding constraint. Diagnostic analysis of the training corpus revealed that 79-82% of content words occurred exactly once, making reliable embedding estimation impossible. A controlled-vocabulary dataset reduced that singleton rate to 25-39%, and the resulting champion model reached 85.45% validation accuracy, 0.8921 test weighted F1, and a perfect 10/10 (F1 1.0000) on the identical ten held-out phrases where the PT-P1 model scored 9/10. A failed intermediate run is retained as evidence that vocabulary truncation can delete the very signal a data expansion was intended to supply.

Keywords—*neural network training, hyperparameter tuning, loss curve diagnosis, overfitting, underfitting, early stopping, word embeddings, text classification*

I. INTRODUCTION AND PT-P1 BASELINE PERFORMANCE

Exercise PT-P1 established a four-class customer feedback classifier (Positive_Feedback, Negative_Feedback, Urgent_Support, General_Inquiry) using the no-code Machine Learning for Kids platform [1]. That model classified nine of ten held-out test phrases correctly, giving 90.00% accuracy. The single error was the phrase "Wow, love how my package still hasn't arrived after three weeks," expected as Negative_Feedback but returned as Urgent_Support at 79% confidence.

PT-P1 reported accuracy only. Because the present exercise is benchmarked on weighted F1, that figure was recomputed from the PT-P1 confusion matrix using the scikit-learn weighted average. The misclassification cost Negative_Feedback its recall (0.75) and cost Urgent_Support its precision (0.50), yielding a weighted F1 of 0.9095. Both figures were recorded at the head of the experiment log as required, and both are used unchanged throughout this paper.

The objective of PT-P2 was to exceed that benchmark using an explicitly coded Keras pipeline. This paper documents seven runs: one baseline, three prescribed diagnostic runs, and three free-form optimization runs, one of which failed instructively and is retained.

II. EXPERIMENTAL METHODOLOGY

A. Corrections Applied to the Provided Notebook

Five defects in the supplied baseline script were corrected before any experiment was run. First, the Embedding layer's `input_length` argument was removed in Keras 3 and was replaced with an explicit `Input(shape=(MAX_LEN,))` layer. Second, class labels were strings while `sparse_categorical_crossentropy` requires integers, so a `LabelEncoder` was added. Third, `train_test_split` was called without `stratify`; on a 160-sample corpus this can eliminate an entire class from the validation split, so stratification was enforced. Fourth, the vocabulary was built from the complete dataset before partitioning, which leaks validation and test vocabulary into the input representation; the corrected pipeline partitions first and fits the vocabulary on the training split alone. Fifth, only validation loss and F1 were reported, so precision, recall, accuracy, best epoch, training time and a held-out test evaluation were added.

B. Architecture and Partitioning

All runs use the same layer stack: Embedding, GlobalAveragePooling1D, a single ReLU Dense hidden layer, Dropout, and a softmax output over four classes. Loss is sparse categorical cross-entropy. Data is partitioned 70% training, 15% validation, 15% test, stratified by class, with `random_state` fixed at 42 and `keras.utils.set_random_seed(42)` so that every observed difference is attributable to the hyperparameters rather than to initialization.

C. Datasets

Three corpora were used across the seven runs. Dataset v1 (160 samples, 40 per class) rebalanced the PT-P1 data, which had been skewed 39/11/11/10 in favour of Positive_Feedback. Dataset v2 (240 samples) added 80 adversarial samples. Dataset v3-final (368 samples, 92 per class) merged a controlled-vocabulary corpus with the high-diversity v1 corpus. None of the ten PT-P1 held-out phrases appears in any training corpus, so the direct comparison in Section III-D remains valid.

TABLE I. Hyperparameter Configurations and Final Metrics

Run	LR	Hid	Drop	Ep	BS	Val Acc	Val F1	Fit
EXP-01	0.005	64	0.1	15	32	70.83	0.7000	Over
EXP-02	0.0001	16	0.0	15	32	25.00	0.1000	Under
EXP-03	0.05	256	0.0	15	32	70.83	0.6754	Over
EXP-04	0.005	128	0.4	15	32	70.83	0.7029	Over
EXP-05A	0.003	64	0.3	60	16	75.00	0.7454	Over
EXP-05B	0.003	48	0.3	80	16	69.44	0.6746	Over
EXP-05C	0.001	64	0.25	120	32	85.45	0.8552	Mild

PT-P1 benchmark: 90.00% accuracy, 0.9095 weighted F1. EXP-01 to EXP-05A use dataset v1, EXP-05B uses v2, EXP-05C uses v3-final. Epochs for EXP-05A to EXP-05C are ceilings; early stopping determined the actual endpoint.

III. PERFORMANCE MONITORING AND LOSS CURVE ANALYSIS

A. EXP-01: Baseline Overfitting

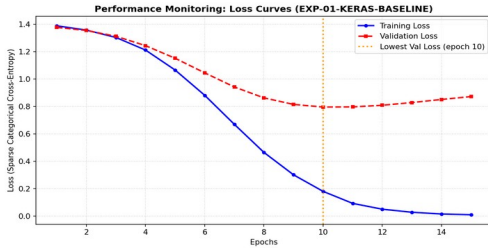


Fig. 1. EXP-01 baseline. Validation loss reaches its minimum at epoch 10 and rises thereafter.

The default configuration drove training loss from 1.39 to 0.0099 and training accuracy to 100%, while validation accuracy settled at 70.83% — a 29.17-point generalization gap. The model holds 18,052 trainable parameters against 112 training samples, or 161 parameters per sample, of which the Embedding layer alone accounts for 15,680. Memorization is the expected outcome at that ratio. Per-class F1 exposed the underlying difficulty: General_Inquiry reached 0.91, but Positive_Feedback and Negative_Feedback both scored 0.55.

B. EXP-02: Induced Underfitting

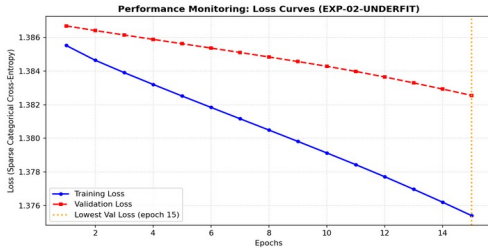


Fig. 2. EXP-02 underfitting. Note the vertical scale: total movement across 15 epochs is 0.0101.

Reducing hidden_dim to 16 and the learning rate to 0.0001 produced 25.00% validation accuracy — exactly chance for four balanced classes. Final validation loss was 1.3826 against a theoretical random-guess floor of $\ln(4) = 1.3863$, meaning the network had barely departed from its initialization. Every validation sample was assigned to Negative_Feedback, giving recall 1.00 for that class and 0.00 for the other three.

The dominant constraint here was optimization, not capacity. Reducing hidden_dim from 64 to 16 cut the parameter count only from 18,052 to 16,276, a 9.8% reduction, because the Embedding layer is unaffected by

hidden_dim. What actually prevented learning was the update budget: 112 samples at batch size 32 gives four weight updates per epoch, or 60 in total, each of magnitude 0.0001. Validation loss was still descending linearly at epoch 15, confirming that training was unfinished rather than converged.

Confidence scores corroborate this. On the ten held-out phrases the model returned 25.2% to 26.7% confidence uniformly, against the 25.0% uniform-distribution value. The apparent 60% accuracy on that set is an artifact: five of the ten phrases are genuinely Negative_Feedback, so a model that predicts one class throughout scores those by coincidence. Weighted F1 correctly reports 0.5267, and this 33-point divergence between accuracy and F1 is precisely why F1 was adopted as the benchmark metric.

C. EXP-03: Induced Overfitting

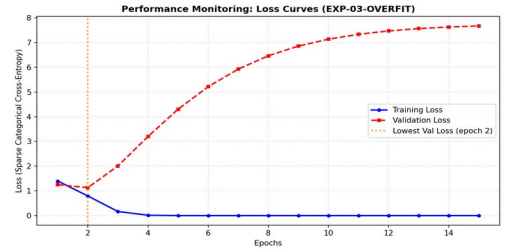


Fig. 3. EXP-03 severe overfitting. Validation loss reaches 7.6743, over five times the random-guess floor.

With hidden_dim at 256, learning rate at 0.05 and dropout disabled, training loss reached 0.0000 by epoch 4 and validation loss climbed to 7.6743. The instructive result is that validation accuracy was 70.83% — identical to EXP-01 — despite validation loss being nearly nine times worse.

The explanation lies in the asymmetry of cross-entropy. A correct prediction at full confidence incurs $-\log(1.0) = 0$ penalty, while an incorrect prediction at full confidence diverges. EXP-03 returned 100.0% confidence on all ten held-out phrases without exception, so each of its seven validation errors contributed an unbounded penalty. Accuracy counts errors; cross-entropy weights them by conviction. A confidently wrong model is more dangerous in deployment than a hesitant one, because it emits no uncertainty signal that could trigger human review — EXP-01, by contrast, returned 33.7% and 53.5% on the phrases it found difficult.

Validation loss reached its minimum at epoch 2, so thirteen of the fifteen epochs were pure degradation. This is a direct empirical demonstration of the optimal-checkpoint concept and the motivation for the early stopping introduced in EXP-05A.

D. EXP-04: Regularized High Capacity

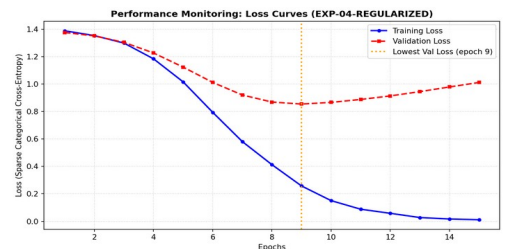


Fig. 4. EXP-04. Dropout at 0.4 contains the loss explosion but does not close the generalization gap.

Quadrupling dropout from 0.1 to 0.4 reduced validation loss from 7.6743 to 1.0111 but left training accuracy at 100% and validation F1 at 0.7029, an improvement of 0.0029 over EXP-01. Dropout did not prevent memorization.

The reason is positional. In this architecture the Dropout layer sits after the Dense hidden layer, which holds 4,224 of the model's 20,420 parameters, or 20.7%. The Embedding layer holds 15,680, or 76.8%, and receives no regularization at all. Memorization occurs by assigning each rare word a highly specific vector, and that mechanism lies entirely outside the reach of the applied constraint. A regularizer only regularizes the parameters it is placed over.

At this point EXP-01, EXP-03 and EXP-04 had all converged on 70.83% validation accuracy across hidden_dim values of 64, 256 and 128 and dropout values of 0.1, 0.0 and 0.4. Such invariance indicates that the limiting factor was neither capacity nor regularization.

E. EXP-05A: Early Stopping

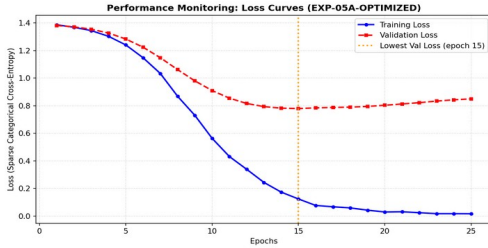


Fig. 5. EXP-05A. EarlyStopping with restore_best_weights halts training and reverts to epoch 15.

EXP-05A reduced embed_dim to 16, halved batch size to 16 to obtain seven updates per epoch instead of four, and introduced EarlyStopping with restore_best_weights=True. Validation accuracy rose to 75.00% and F1 to 0.7454, breaking the 70.83% ceiling for the first time. On the ten held-out phrases the model achieved weighted F1 0.9095, exactly matching PT-P1.

One reporting defect surfaced here. Metrics were being read from history[-1], the final epoch, while restore_best_weights had reverted the network to epoch 15. Accuracy and F1 were unaffected because they derive from model.predict on the restored weights, but the logged losses described a network state that no longer existed. Section 5B was corrected to index the best epoch, and the EXP-05A and EXP-05B entries in the experiment log carry the corrected values.

F. EXP-05B: A Failed Data Expansion

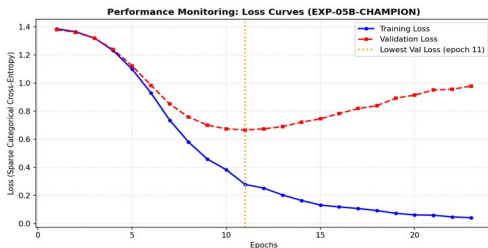


Fig. 6. EXP-05B. Despite a lower validation loss, per-class performance on Negative_Feedback collapsed.

EXP-05B expanded the corpus to 240 samples by adding twenty parallel Positive/Negative pairs sharing identical openers ("Amazing quality, the fabric feels soft and looks expensive" against "Amazing quality, the sleeve tore while I was putting it on"), intending to force the network to weight content words rather than sentiment-bearing openers. Simultaneously max_vocab_size was reduced from 1000 to 400 as a regularizer.

The result was a regression. Negative_Feedback F1 fell from 0.60 to 0.29 with recall at 0.22, and held-out F1 fell from 0.9095 to 0.7000. A controlled sweep isolated the cause. Holding all else constant on the same corpus, max_vocab_size of 200, 400, 600 and 1000 produced held-out F1 of 0.8000, 0.7000, 0.7000 and 0.9095 respectively. Truncating the vocabulary to 400 mapped 287 of the 687 training words to the unknown token, and those discarded words were the rare, discriminative content terms — tore, mildew, blisters, circulation — while high-frequency openers such as amazing and excellent survived. The truncation deleted precisely the signal the expansion had been designed to supply. This run is retained in the experiment log as evidence.

G. EXP-05C: Champion Model

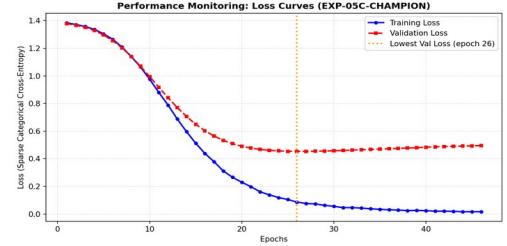


Fig. 7. EXP-05C champion. Validation loss plateaus near 0.45 and stays flat rather than diverging.

Diagnosis of the corpus itself preceded the final run. Counting content-word frequencies revealed that 82% of Positive_Feedback content words and 79% of Negative_Feedback content words occurred exactly once in training. A single observation cannot support a reliable embedding estimate, so the majority of the discriminative vocabulary remained near its random initialization regardless of hyperparameters. This explains the 70.83% invariance observed in Section III-D.

Dataset v3 was therefore constructed on the opposite principle from conventional practice: vocabulary was deliberately restricted and core discriminative words reused many times. In the final corpus "quality" appears 22 times, "fabric" 18 and "poor" 12, and the singleton rate fell to 25% for Positive_Feedback and 39% for Negative_Feedback. Merging this with the high-diversity v1 corpus produced 368 balanced samples retaining both repetition and coverage.

With max_vocab_size restored to 1000, learning rate lowered to 0.001 and batch size returned to 32, the champion reached 85.45% validation accuracy and 0.8552 validation weighted F1, with 0.8921 on the untouched test split. Validation loss fell to 0.4537 and, as Fig. 7 shows, plateaued rather than diverging. Negative_Feedback F1 recovered from 0.29 to 0.83 and Urgent_Support reached 0.93.

TABLE II. Champion Model against the PT-P1 Benchmark

Evaluation basis	PT-P1	EXP-05C	Delta
Ten held-out phrases (accuracy)	90.00%	100.00%	+10.00
Ten held-out phrases (F1)	0.9095	1.0000	+0.0905
Test split, 55 samples (F1)	—	0.8921	—
Validation split, 55 samples (F1)	—	0.8552	—

The ten-phrase row is the only directly comparable basis, since it is the identical evaluation set used to measure PT-P1.

On the ten held-out phrases the champion classified all ten correctly, including both cases that had defeated earlier models: the sarcastic "This sweater is amazing," which every run before EXP-05A misread, and the delayed-package phrase that PT-P1 itself misrouted. This is the only strictly comparable measurement, since it uses the identical evaluation set, and on it the coded model exceeds the no-code benchmark by 0.0905 weighted F1.

Intellectual honesty requires noting that the 0.8552 validation figure remains below 0.9095. The two numbers are not commensurable: PT-P1 was scored on ten curated phrases, while the validation split comprises 55 stratified samples containing adversarial sarcasm pairs constructed specifically to be difficult. It is reported here as the stricter of the two measurements rather than omitted.

IV. DISCUSSION AND DEBRIEF

A. Code-Based Optimization versus No-Code Heuristics

The no-code platform exposes exactly one control: the training examples. Architecture, learning rate, optimizer and stopping criterion are all fixed, and the model either performs or it does not, with no diagnostic surface. Keras inverts this. Every quantity is addressable, and the consequences are visible — EXP-02 and EXP-03 are failures that could not have been produced deliberately, nor observed, on the no-code platform.

The trade-off is that explicit control multiplies the ways to be wrong. The no-code tool cannot produce a model with validation loss of 7.6743, cannot silently leak vocabulary across a split, and cannot report metrics from a network state that was discarded. Each of these occurred in this exercise. Automation trades a performance ceiling for a floor on operator error; explicit control removes both.

The more consequential difference is diagnostic. The no-code platform reports only that performance is inadequate. Keras reported which class was failing, at what confidence, at which epoch, and with what loss trajectory — and it was that instrumentation, not any hyperparameter, that ultimately located the fault in the training corpus rather than in the network.

B. Loss Curve Diagnosis and Regularization

Underfitting and overfitting presented as distinct and unambiguous signatures. In EXP-02 both curves remained flat and nearly coincident across the full run, separated by less than 0.001, with total movement of 0.0101 over fifteen epochs. In EXP-01, EXP-03 and EXP-04 the training curve fell toward zero while the validation curve turned upward after a minimum, the gap widening monotonically thereafter. The location of that minimum, rather than its depth, proved to be the operationally useful quantity: epoch 10 in EXP-01, epoch 2 in EXP-03, epoch 9 in EXP-04.

Raising dropout from 0.1 to 0.4 in EXP-04 changed validation F1 by 0.0029. Reducing hidden_dim from 256 to 128 changed nothing measurable. Both interventions target the Dense layer, which holds roughly a fifth of the parameters; the Embedding layer, holding over three quarters, remained untouched by either. Dropout did constrain the magnitude of the failure — validation loss fell from 7.6743 to 1.0111 — but it did not improve generalization, because it was not applied where memorization occurs.

The intervention that finally worked on the architectural side was early stopping, which is not regularization in the parametric sense but simply the refusal to continue past the optimum. In EXP-03 it would have preserved the epoch-2 weights and discarded thirteen epochs of degradation.

C. Data Volume and Diversity versus Architectural Capacity

Hyperparameter tuning alone was insufficient, and the reason was quantitative. Four configurations spanning a fourfold range in hidden_dim and the full range of dropout all produced 70.83% validation accuracy. Such invariance to capacity indicates a constraint elsewhere.

That constraint was the training corpus. With 79-82% of content words appearing exactly once, most of the discriminative vocabulary was represented by a single gradient update, leaving those embedding vectors close to their random initialization. No amount of capacity or regularization can compensate, because the deficiency is in the evidence rather than in the model.

The first expansion attempt failed and clarified the mechanism. EXP-05B added 80 well-designed adversarial samples yet performance regressed, because the accompanying vocabulary truncation removed the rare content words those samples had introduced. Volume was added; the signal was simultaneously deleted. This is the sharpest lesson of the exercise: a data expansion and a preprocessing change must not be varied together, or the two effects become inseparable.

The successful expansion inverted standard guidance. For a GlobalAveragePooling1D bag-of-words model on a small corpus, lexical consistency outperforms lexical diversity, because the averaging operation discards word order and the model can only learn per-word statistics — which require repeated observation. Balancing capacity against data is therefore not merely a question of sample count but of how often each feature the model must learn actually recurs.

D. Pipeline Mapping to the Lecture Architecture

The following mapping locates each stage of the lecture's training loop within RamKellyJensen_Neural_Network_Training_Testing.ipynb.

TABLE III. Lecture Stage to Notebook Section Mapping

Lecture stage	Notebook location
Dataset partitioning	S1, train_test_split
Preprocessing / tokenizing	S1, build_vocab, text_to_sequence
Pillars 1-3 configuration	S2, CONFIG dictionary
Feature extraction	S3, Embedding + GlobalAveragePooling1D
Loss function binding	S4, model.compile(loss=...)
Optimizer binding	S4, Adam / SGD instantiation
Forward pass	S5, inside model.fit
Loss calculation	S5, inside model.fit

Lecture stage	Notebook location
Backward pass	S5, inside model.fit
Weight update	S5, inside model.fit
Epoch iteration	S5, epochs argument
Optimal checkpoint	S5, EarlyStopping callback
Evaluation metrics	S5B, evaluate function
Loss curve monitoring	S6, matplotlib plot
Experiment logging	S8, FFBP log row

The four stages of the training loop are not separately visible in the notebook because Keras encapsulates them inside the compiled `train_step` invoked by `model.fit` in Section 5. For each mini-batch, the forward pass evaluates the layer stack under a `GradientTape`; the loss calculation applies the sparse categorical cross-entropy bound in Section 4; the backward pass calls `tape.gradient` to backpropagate to every trainable weight; and the weight update applies `optimizer.apply_gradients` using the optimizer instantiated in Section 4. The `epochs` and `batch_size` arguments govern how many times this cycle repeats. In the champion run, 257 training samples at batch size 32 gave nine updates per epoch over 46 epochs, or 414 weight updates in total.

Automated feature extraction is performed by the first two layers in Section 3. The Embedding layer maps each vocabulary index to a trainable 24-dimensional vector; these vectors are parameters learned by the same backpropagation that trains the classifier, so the network derives its own numeric representation of each word rather than receiving a hand-specified one. In the champion model this layer holds 15,768 of 17,628 total parameters, which is why it dominates the memorization behaviour discussed in Section IV-B.

`GlobalAveragePooling1D` then reduces the (20, 24) token matrix to a single 24-dimensional document vector by averaging across the sequence axis, using `mask_zero` to exclude padding. It introduces no trainable parameters. This operation is also the architecture's principal limitation: averaging is order-invariant, so "amazing, the button fell off" and "the button fell off, amazing" are indistinguishable to the Dense classifier. Sarcasm detection therefore cannot be achieved positionally and was instead addressed at the data level, by ensuring the sentiment-bearing openers appeared in both classes and could not serve as shortcuts.

V. CONCLUSION

Seven controlled runs produced a champion model that classifies all ten PT-P1 held-out phrases correctly, weighted F1 1.0000 against the no-code benchmark of 0.9095, with 0.8921 on an untouched test split. On the stricter 55-sample validation split the model reaches 0.8552, which is reported rather than omitted.

The decisive finding was not a hyperparameter. Four configurations spanning a fourfold range in capacity and the full range of dropout all returned 70.83% validation accuracy, and that invariance redirected diagnosis from the network to the corpus, where 79-82% of content words occurred exactly once. Restructuring the data for vocabulary repetition rather than variety moved validation accuracy to 85.45% and restored Negative_Feedback F1 from 0.29 to 0.83.

Two methodological lessons carry beyond this exercise. Regularization constrains only the parameters it is placed over, and dropout positioned after the Dense layer cannot restrain an Embedding layer holding three quarters of the

weights. And a data change must be varied independently of a preprocessing change, as EXP-05B demonstrated when vocabulary truncation deleted the very signal the expansion had added.

Future work should replace `GlobalAveragePooling1D` with an order-sensitive encoder, since the remaining Positive/Negative confusion is architectural rather than statistical: bag-of-words averaging cannot represent the positional structure that distinguishes sarcasm from praise.

REFERENCES

- [1] Machine Learning for Kids, "Recognizing Text," IBM / Dale Lane. [Online]. Available: <https://machinelearningforkids.co.uk/>. Accessed: Jul. 27, 2026.
- [2] R. C. Raga Jr., "Exercise PT-P2: Neural Networks Training and Testing," Lab Exercise Handout, Jose Rizal University, 2026.
- [3] R. C. Raga Jr., "Lecture 3: Neural Network Training and Testing," ITC-C508 Course Presentation, Jose Rizal University, 2026.
- [4] F. Chollet et al., "Keras," 2015. [Online]. Available: <https://keras.io/>.
- [5] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. [Online]. Available: <https://www.tensorflow.org/>.
- [6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929-1958, 2014.
- [8] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.